



INSTITUTO PROFESIONAL
SAN SEBASTIAN

PROGRAMACIÓN PARA LA CIENCIA DE DATOS

SESIÓN SINCRÓNICA TEÓRICA 3

DEPURACIÓN Y OPTIMIZACIÓN DE ALGORITMOS PARA ANÁLISIS DE DATOS

**INSTITUTO PROFESIONAL
ACREDITADO**



**GESTIÓN INSTITUCIONAL
DOCENCIA DE PREGRADO**
3 AÑOS / HASTA MARZO 2025



- **Desarrollo de temas de la Unidad N°3:**
 - ✓ **Tema 1: Marco conceptual: la triada de la calidad.**
 - ✓ **Tema 2: Depuración con método**
 - ✓ **Tema 3: Robustez: manejo de excepciones.**
 - ✓ **Tema 4: Medición: Profiling antes de optimizar**
 - ✓ **Tema 5: Optimización práctica: pandas y numpy**
 - ✓ **Tema 6: Validación: pruebas de regresión**
 - ✓ **Tema 7: Refactorización y estilo.**

Introducción

En esta presentación tiene como objetivo presentar el proceso de **Depuración y optimización de algoritmos para el análisis de datos**.

En esta presentación veremos cómo llevar código “que funciona” a código **confiable, eficiente y mantenible**. Partiremos de problemas reales (errores sutiles, tiempos altos, caídas por excepciones) y aplicaremos un flujo disciplinado: **medir → corregir → probar → optimizar → validar → documentar**. Usaremos *profiling*, manejo de excepciones y vectorización (NumPy/pandas), cerrando con pruebas de regresión y buenas prácticas de refactorización.



UNIDAD 3



RESUMENES GRÁFICOS

Marco Conceptual: La Tríada de la Calidad

En programación para ciencia de datos, tres pilares fundamentales sostienen cualquier proyecto de calidad. Dominar estos principios te permitirá crear código profesional, sostenible y escalable.



Correctitud

Asegura resultados exactos, reproducibles y reproducibles y completamente trazables en trazables en cada ejecución.



Eficiencia

Optimiza el uso de tiempo y memoria, garantizando escalabilidad en datasets crecientes.



Mantenibilidad

Código claro, modular y documentado que facilita pruebas, extensiones y colaboración.



Depuración con Método

La depuración efectiva no es arte de magia, sino aplicación sistemática de técnicas probadas. Un enfoque metódico reduce drásticamente el tiempo de resolución de errores.

01

Aislar la causa raíz

Utiliza breakpoints y walkthroughs para identificar el origen exacto del problema. Evita aplicar parches sin comprender la causa subyacente.

03

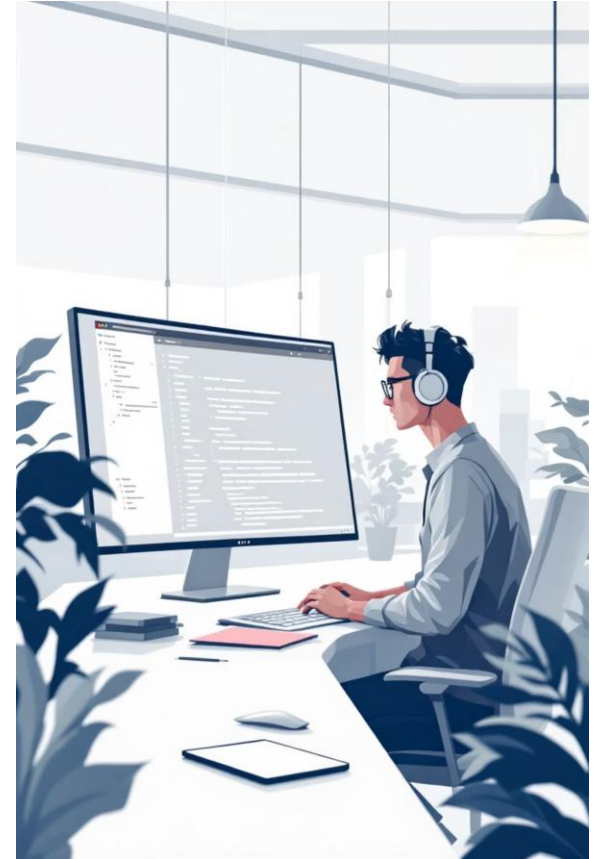
Logging estratégico

Implementa niveles INFO/WARN/ERROR con mensajes accionables que permitan trazabilidad completa y diagnósticos precisos.

02

Inspección dirigida

Verifica variables y estados clave en momentos críticos. Comprueba que las pre-condiciones y post-condiciones se cumplan correctamente.



Estrategia defensiva

Un código robusto anticipa fallos previsibles y responde de manera controlada, asegurando continuidad operativa sin sacrificar transparencia.

- Captura excepciones específicas como como `FileNotFoundException` o `ValueError`
- Define valores por defecto seguros para mantener la operación
- Registra cada incidente para auditoría auditoría posterior

Mejores prácticas

❏ **Evita capturas genéricas** que oculten problemas reales. Un simple `except Exception` puede enmascarar errores críticos.

Siempre proporciona mensajes claros al usuario y deja evidencia detallada en logs para facilitar el diagnóstico y la corrección.



Medición: Profiling Antes de Optimizar

La optimización prematura es la raíz de muchos males. Mide primero, optimiza después. Las herramientas de profiling revelan dónde realmente importa tu esfuerzo.

1

Identificar cuellos de botella

Utiliza `cProfile`, `line_profiler` o `timeit` para detectar las funciones que consumen más recursos computacionales.

2

Priorizar por impacto

Enfoca tus esfuerzos en las funciones críticas que realmente afectan el rendimiento global del sistema.

3

Establecer línea base

Define métricas de referencia claras para poder comparar y validar las mejoras introducidas.



Optimización Práctica: pandas y NumPy

La verdadera eficiencia en ciencia de datos viene de aprovechar las operaciones vectorizadas y comprender las estructuras de datos subyacentes.

Vectorización inteligente

Reemplaza bucles explícitos por operaciones operaciones columnares nativas. Evita `.apply()` cuando existan alternativas vectorizadas más eficientes.

- Operaciones aritméticas directas sobre columnas
- Métodos nativos de pandas/NumPy

Agregaciones correctas

Domina `groupby` y `agg` para transformaciones complejas. Valida cardinalidades en merges para prevenir duplicaciones inesperadas.

- Verificar llaves de join
- Evitar doble conteo en agregaciones

Optimización de memoria

Usa tipos eficientes como `category`, `int32` o `float32`. Elimina trabajo redundante evitando recargas y transformaciones repetidas.

- Reducir huella de memoria
- Cachear cálculos costosos



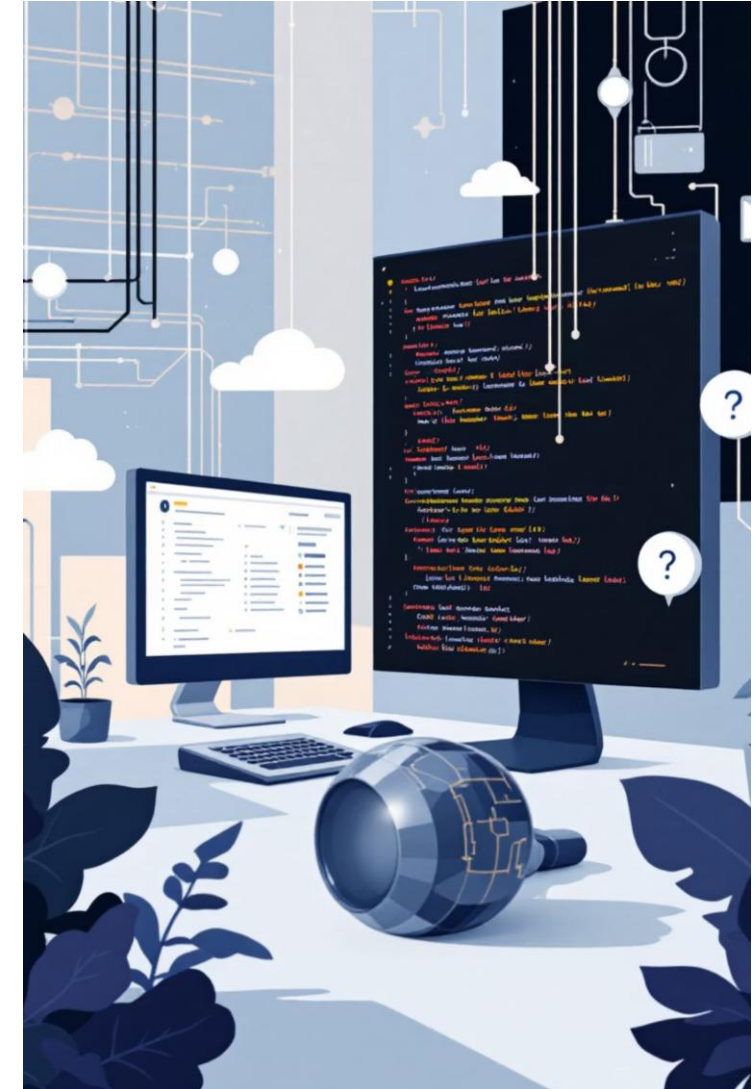
Validación: Pruebas de Regresión

Las optimizaciones sin validación son experimentos peligrosos. Las pruebas de regresión garantizan que tus mejoras no introduzcan nuevos errores.

1 Crea casos de prueba con oráculos conocidos y define tolerancias numéricas apropiadas para comparaciones flotantes.

2 Equivalencia semántica
Compara resultados 'antes vs. después' para verificar que la optimización mantiene el comportamiento esperado.

3 Casos de borde
Prueba situaciones límite: valores NaN, primer elemento de grupo, filas atípicas y datasets vacíos.



Refactorización y Estilo

El código limpio no es un lujo, es una inversión. La refactorización sistemática mejora la colaboración, reduce errores y acelera el desarrollo futuro.

Arquitectura clara

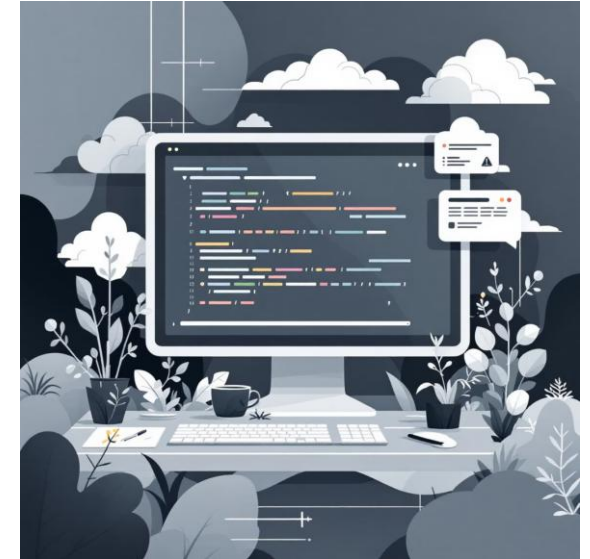
Separa responsabilidades: cargar → limpiar → transformar → reportar. Las funciones puras sin efectos secundarios facilitan el testing y la depuración.

Tipado y documentación

Utiliza type hints opcionales y documenta parámetros, retornos y efectos. El código debe ser autoexplicativo.

Principio DRY

Don't Repeat Yourself. Encapsula lógica repetida en funciones reutilizables con nombres descriptivos y docstrings completos.



 **Regla de oro:** Si necesitas comentarios para explicar *qué* hace el código, probablemente necesitas mejores nombres de variables y funciones.



Reflexión participativa

- ¿Qué priorizar si el resultado es correcto, pero no cumple el SLA?
- Comparte una estrategia para demostrar equivalencia antes/después.
- Propón una regla de logging que evite ruido y ayude a diagnosticar.



Síntesis

- Workflow: Medir → Corregir → Probar → Optimizar → Validar → Documentar
- Decisiones basadas en evidencia (profiling + tests)
- Resultados reproducibles, escalables y mantenibles



INSTITUTO PROFESIONAL
SAN SEBASTIAN

PROGRAMACIÓN PARA LA CIENCIA DE DATOS

SESIÓN SINCRÓNICA TEÓRICA 3

DEPURACIÓN Y OPTIMIZACIÓN DE ALGORITMOS PARA ANÁLISIS DE DATOS

**INSTITUTO PROFESIONAL
ACREDITADO**



**GESTIÓN INSTITUCIONAL
DOCENCIA DE PREGRADO**
3 AÑOS / HASTA MARZO 2025